Complete Python Learning Guide

Table of Contents

1. Introduction
2. Getting Started
3. Fundamentals
4. Data Structures
5. Control Flow
6. Functions
7. Object-Oriented Programming
8. File Handling & I/O
9. Modules & Packages
10. Error Handling
11. Common Libraries
12. Best Practices
13. Project Ideas

Introduction

Python is a high-level, interpreted programming language known for its simplicity and readability. It's widely used in web development, data science, automation, artificial intelligence, and more.

Why Python?

- Easy to learn and read
- Large ecosystem of libraries
- Versatile applications
- Strong community support
- Great for beginners and professionals alike

Getting Started

Installation

Windows/Mac/Linux:

1. Visit python.org
2. Download the latest version (3.10 or higher)
3. Run the installer
4. Verify installation: `python --version`

Your First Program

```python
print("Hello, World!")
```

Run it: `python filename.py`

Interactive Mode

Type `python` in your terminal to enter interactive mode. Great for testing quick code snippets.

Fundamentals

Variables and Data Types

```python
# Variables (no type declaration needed)
name = "Luis"
age = 20
gpa = 3.9
is_student = True

# Basic data types
integer = 42
floating_point = 3.14
string = "Python is awesome"
boolean = False

print(type(integer))  # <class 'int'>
print(type(floating_point))  # <class 'float'>
print(type(string))  # <class 'str'>
print(type(boolean))  # <class 'bool'>
```

String Operations

python

```python
# String concatenation
greeting = "Hello" + " " + "World"

# String formatting
name = "Luis"
message = f"My name is {name}"  # f-string (modern approach)
message = "My name is {}".format(name)  # .format() method
message = "My name is %s" % name  # old-style (avoid)

# Common string methods
text = "python"
print(text.upper())  # PYTHON
print(text.capitalize())  # Python
print(text.replace("python", "java"))  # java
print("hello world".split())  # ['hello', 'world']
print("-".join(["a", "b", "c"]))  # a-b-c
```

Input and Output

python

```python
# Getting user input
name = input("What's your name? ")
print(f"Hello, {name}!")

# Multiple outputs
print("Hello", "World", sep="-", end="!\n")  # Hello-World!

# Type conversion
age = int(input("Enter your age: "))
price = float(input("Enter price: "))
```

Basic Arithmetic

python

```python
addition = 5 + 3  # 8
subtraction = 5 - 3  # 2
multiplication = 5 * 3  # 15
division = 15 / 3  # 5.0 (always returns float)
floor_division = 15 // 3  # 5 (integer division)
modulo = 17 % 5  # 2 (remainder)
exponentiation = 2 ** 3  # 8 (power)
```

Data Structures

Lists (Ordered, Mutable)

python

```python
# Creating lists
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
empty = []

# Accessing elements (0-indexed)
first = numbers[0]  # 1
last = numbers[-1]  # 5 (negative indexing)

# Slicing
slice1 = numbers[1:4]  # [2, 3, 4] (start inclusive, end exclusive)
slice2 = numbers[:3]  # [1, 2, 3]
slice3 = numbers[::2]  # [1, 3, 5] (every 2nd element)

# List methods
numbers.append(6)  # Add to end
numbers.insert(0, 0)  # Insert at position
numbers.remove(3)  # Remove specific value
popped = numbers.pop()  # Remove and return last element
numbers.sort()  # Sort in place
sorted_numbers = sorted(numbers)  # Return new sorted list
numbers.reverse()  # Reverse in place
length = len(numbers)

# List comprehension (elegant way to create lists)
squares = [x**2 for x in range(5)]  # [0, 1, 4, 9, 16]
evens = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]
```

Tuples (Ordered, Immutable)

python

```python
# Creating tuples
coordinates = (10, 20)
single = (1,)  # Note the comma for single-element tuple
empty = ()

# Accessing elements (same as lists)
x = coordinates[0]  # 10

# Tuples are immutable (cannot change after creation)
# coordinates[0] = 5  # This would cause an error

# Unpacking
x, y = coordinates  # x = 10, y = 20

# Tuple methods
numbers = (1, 2, 3, 2, 1)
count = numbers.count(2)  # 2
index = numbers.index(3)  # 2
```

Dictionaries (Unordered, Key-Value Pairs)

python

```python
# Creating dictionaries
person = {"name": "Luis", "age": 20, "city": "Houston"}
student = dict(name="Luis", major="Finance")

# Accessing values
name = person["name"]  # "Luis"
name = person.get("name")  # "Luis" (safer, returns None if key doesn't exist)
name = person.get("nickname", "N/A")  # "N/A" (default value)

# Adding/Updating
person["gpa"] = 3.9
person.update({"age": 21, "school": "UH"})

# Removing
del person["city"]
removed_value = person.pop("age")

# Dictionary methods
keys = person.keys()  # dict_keys(['name', 'gpa', 'school'])
values = person.values()  # dict_values(['Luis', 3.9, 'UH'])
items = person.items()  # dict_items([('name', 'Luis'), ...])

# Dictionary comprehension
squares_dict = {x: x**2 for x in range(5)}  # {0: 0, 1: 1, 2: 4, ...}

# Iterating through dictionaries
for key in person:
    print(f"{key}: {person[key]}")

for key, value in person.items():
    print(f"{key}: {value}")
```

Sets (Unordered, Unique Values)

python

```python
# Creating sets
numbers = {1, 2, 3, 4, 5}
unique = set([1, 2, 2, 3, 3, 3])  # {1, 2, 3}
empty = set()  # Note: {} creates a dict, not a set

# Set operations
numbers.add(6)
numbers.remove(3)  # Raises KeyError if not found
numbers.discard(10)  # Safe removal, no error if not found

# Set math
set_a = {1, 2, 3}
set_b = {3, 4, 5}
union = set_a | set_b  # {1, 2, 3, 4, 5}
intersection = set_a & set_b  # {3}
difference = set_a - set_b  # {1, 2}
symmetric_diff = set_a ^ set_b  # {1, 2, 4, 5}
```

Control Flow

If/Elif/Else

python

```python
age = 20

if age < 13:
    print("Child")
elif age < 18:
    print("Teenager")
elif age < 65:
    print("Adult")
else:
    print("Senior")

# Ternary operator
status = "Adult" if age >= 18 else "Minor"
```

Loops

For Loops

python

```python
# Looping through a list
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(fruit)

# Using range
for i in range(5):  # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8, 2):  # 2, 4, 6 (start, stop, step)
    print(i)

# Enumerate (get index and value)
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Loop through dictionary
person = {"name": "Luis", "age": 20}
for key, value in person.items():
    print(f"{key}: {value}")
```

While Loops

python

```python
count = 0
while count < 5:
    print(count)
    count += 1

# Break and continue
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break  # Exit the loop
    if user_input.startswith("#"):
        continue  # Skip this iteration
    print(f"You entered: {user_input}")
```

Comparison and Logical Operators

python

```python
# Comparison operators
5 == 5  # True
5 != 3  # True
5 > 3  # True
5 < 3  # False
5 >= 5  # True
5 <= 4  # False

# Logical operators
True and True  # True
True or False  # True
not True  # False

# Chaining comparisons
3 < 5 < 10  # True (equivalent to 3 < 5 and 5 < 10)

# Membership testing
5 in [1, 2, 3, 4, 5]  # True
"a" not in "hello"  # True
```

Functions

Defining and Calling Functions

python

```python
# Basic function
def greet():
    print("Hello!")

greet()

# Function with parameters
def greet_person(name):
    print(f"Hello, {name}!")

greet_person("Luis")

# Function with return value
def add(a, b):
    return a + b

result = add(5, 3)  # 8

# Multiple return values
def divide_with_remainder(a, b):
    quotient = a // b
    remainder = a % b
    return quotient, remainder

q, r = divide_with_remainder(17, 5)  # q = 3, r = 2
```

Default Parameters and Keyword Arguments

```python
```

```python
# Default parameters
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Luis")  # Hello, Luis!
greet("Luis", "Hi")  # Hi, Luis!

# Keyword arguments
def describe_person(name, age, city="Unknown"):
    print(f"{name} is {age} years old and lives in {city}")

describe_person("Luis", 20, city="Houston")
describe_person(name="Luis", age=20, city="Houston")
describe_person(age=20, name="Luis")  # Order doesn't matter with kwargs
```

Variable-Length Arguments

python

```python
# *args for variable number of positional arguments
def print_all(*args):
    for item in args:
        print(item)

print_all(1, 2, 3, "hello", 4.5)

# **kwargs for variable number of keyword arguments
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Luis", age=20, city="Houston")

# Combining all types
def flexible_function(a, b, *args, **kwargs):
    print(f"a: {a}, b: {b}")
    print(f"args: {args}")
    print(f"kwargs: {kwargs}")
```

Lambda Functions (Anonymous Functions)

python

```python
# Simple function
square = lambda x: x ** 2
print(square(5))  # 25

# Used with built-in functions
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))  # [1, 4, 9, 16, 25]
evens = list(filter(lambda x: x % 2 == 0, numbers))  # [2, 4]

# Sorting with lambda
students = [("Luis", 3.9), ("Ana", 3.8), ("Bob", 3.7)]
sorted_by_gpa = sorted(students, key=lambda x: x[1], reverse=True)
```

Docstrings and Comments

python

```python
# Comments
# This is a single-line comment

"""
This is a multi-line comment or docstring.
It can span multiple lines.
"""

def calculate_area(radius):
    """
    Calculate the area of a circle.

    Args:
        radius (float): The radius of the circle

    Returns:
        float: The area of the circle

    Example:
        >>> calculate_area(5)
        78.53981633974483
    """
    import math
    return math.pi * radius ** 2
```

Object-Oriented Programming

Classes and Objects

python

```python
class Person:
    # Class variable (shared by all instances)
    species = "Homo sapiens"

    # Constructor (initializer)
    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

    # Instance methods
    def introduce(self):
        return f"Hello, I'm {self.name} and I'm {self.age} years old"

    def have_birthday(self):
        self.age += 1
        print(f"{self.name} is now {self.age}")

    # String representation
    def __str__(self):
        return f"Person(name='{self.name}', age={self.age})"

# Creating objects
person1 = Person("Luis", 20)
person2 = Person("Ana", 19)

# Accessing attributes
print(person1.name)  # Luis
print(person1.species)  # Homo sapiens

# Calling methods
print(person1.introduce())  # Hello, I'm Luis and I'm 20 years old
person1.have_birthday()  # Luis is now 21
```

Inheritance

python

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return "Some generic sound"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed

    # Override parent method
    def make_sound(self):
        return "Woof!"

    def fetch(self):
        return f"{self.name} is fetching!"

dog = Dog("Buddy", "Golden Retriever")
print(dog.make_sound())  # Woof!
print(dog.fetch())  # Buddy is fetching!
```

Encapsulation (Private and Protected Attributes)

python

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute (name mangling)
        self._interest_rate = 0.05  # Protected attribute (convention)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return self.__balance
        return "Invalid amount"

    def withdraw(self, amount):
        if amount > 0 and amount <= self.__balance:
            self.__balance -= amount
            return self.__balance
        return "Invalid amount or insufficient funds"

    # Getter method
    @property
    def balance(self):
        return self.__balance

    # Setter method
    @balance.setter
    def balance(self, amount):
        if amount >= 0:
            self.__balance = amount

account = BankAccount(1000)
account.deposit(500)  # 1500
print(account.balance)  # 1500 (using property)
```

Class Methods and Static Methods

```python
```

```python
class Math:
    pi = 3.14159

    @staticmethod
    def add(a, b):
        """Static method doesn't need instance or class"""
        return a + b

    @classmethod
    def create_circle(cls, radius):
        """Class method receives the class as first argument"""
        return f"Circle with area {cls.pi * radius ** 2}"

# Call static method
result = Math.add(5, 3)  # 8

# Call class method
info = Math.create_circle(5)
```

File Handling & I/O

Reading Files

python

```python
# Reading entire file
with open("data.txt", "r") as file:
    content = file.read()  # Read entire file as string

# Reading line by line
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())  # strip() removes newline

# Reading all lines into a list
with open("data.txt", "r") as file:
    lines = file.readlines()  # List of lines with \n

# Context manager (with statement) automatically closes the file
```

Writing Files

python

```python
# Write mode (overwrites existing file)
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("Line 2\n")

# Append mode (adds to end of file)
with open("output.txt", "a") as file:
    file.write("Line 3\n")

# Writing multiple lines
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

Working with JSON

python

```python
import json

# Convert Python object to JSON string
data = {"name": "Luis", "age": 20, "skills": ["Python", "Finance"]}
json_string = json.dumps(data)  # Convert to JSON string

# Write JSON to file
with open("data.json", "w") as file:
    json.dump(data, file, indent=2)  # indent makes it readable

# Read JSON from file
with open("data.json", "r") as file:
    loaded_data = json.load(file)

# Parse JSON string
parsed = json.loads(json_string)
```

Working with CSV

python

```python
import csv

# Reading CSV
with open("data.csv", "r") as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        print(row)  # Each row is a list

# Reading CSV with headers
with open("data.csv", "r") as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        print(row)  # Each row is a dictionary

# Writing CSV
data = [
    ["Name", "Age", "City"],
    ["Luis", "20", "Houston"],
    ["Ana", "19", "Austin"]
]
with open("output.csv", "w", newline="") as file:
    csv_writer = csv.writer(file)
    csv_writer.writerows(data)
```

Modules & Packages

Built-in Modules

python

```python
# Math module
import math
print(math.pi)  # 3.14159...
print(math.sqrt(16))  # 4.0
print(math.ceil(4.3))  # 5
print(math.floor(4.7))  # 4

# Random module
import random
print(random.randint(1, 10))  # Random integer between 1-10
print(random.choice([1, 2, 3, 4, 5]))  # Random element from list
random.shuffle(my_list)  # Shuffle list in place

# DateTime module
from datetime import datetime, timedelta
now = datetime.now()
print(now.year, now.month, now.day)
tomorrow = now + timedelta(days=1)

# OS module (operating system)
import os
print(os.getcwd())  # Get current directory
files = os.listdir(".")  # List files in directory

# Collections module
from collections import Counter, defaultdict, namedtuple
counter = Counter("hello")  # Count character occurrences
```

Import Methods

python

```python
# Import entire module
import math
result = math.sqrt(16)

# Import specific items
from math import sqrt, pi
result = sqrt(16)
print(pi)

# Import with alias
import numpy as np
from collections import defaultdict as dd

# Import all (generally avoid)
from math import *
```

Creating Your Own Modules

python

```python
# File: math_utils.py
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

PI = 3.14159

# File: main.py
from math_utils import add, multiply, PI
result = add(5, 3)
```

Error Handling

Try-Except Blocks

python

```python
# Basic try-except
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")

# Multiple exception types
try:
    value = int("not a number")
except ValueError:
    print("Could not convert to integer")
except TypeError:
    print("Wrong type")

# Catch all exceptions (avoid in production)
try:
    risky_operation()
except Exception as e:
    print(f"Error occurred: {e}")
```

Try-Except-Else-Finally

python

```python
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found")
else:
    print("File read successfully")
    print(content)
finally:
    # Runs regardless of whether exception occurred
    if 'file' in locals():
        file.close()
```

Raising Exceptions

python

```python
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    if age > 150:
        raise ValueError("Age seems unrealistic")
    return age

try:
    validate_age(-5)
except ValueError as e:
    print(f"Invalid age: {e}")
```

Common Libraries

NumPy (Numerical Computing)

python

```python
import numpy as np

# Creating arrays
arr = np.array([1, 2, 3, 4, 5])
zeros = np.zeros(5)  # [0. 0. 0. 0. 0.]
ones = np.ones(3)  # [1. 1. 1.]
range_arr = np.arange(0, 10, 2)  # [0 2 4 6 8]

# 2D arrays (matrices)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix.shape)  # (2, 3)

# Array operations
arr + 5  # [6 7 8 9 10]
arr * 2  # [2 4 6 8 10]
arr ** 2  # [1 4 9 16 25]

# Statistical functions
np.mean(arr)  # 3.0
np.std(arr)  # Standard deviation
np.max(arr)  # 5
np.min(arr)  # 1
```

Pandas (Data Analysis)

python

```python
import pandas as pd

# Creating DataFrames
data = {"Name": ["Luis", "Ana", "Bob"], "Age": [20, 19, 22]}
df = pd.DataFrame(data)

# Reading data
df = pd.read_csv("data.csv")
df = pd.read_json("data.json")

# Basic operations
print(df.head())  # First 5 rows
print(df.describe())  # Statistical summary
print(df["Name"])  # Select column

# Filtering
young = df[df["Age"] < 21]

# Groupby
by_age = df.groupby("Age").mean()
```

Requests (HTTP Library)

python

```python
import requests

# Making HTTP requests
response = requests.get("https://api.example.com/data")
print(response.status_code)  # 200
print(response.text)  # Response body as string
data = response.json()  # Parse as JSON

# POST request with data
payload = {"username": "luis", "age": 20}
response = requests.post("https://api.example.com/users", json=payload)
```

BeautifulSoup (Web Scraping)

python

```python
from bs4 import BeautifulSoup
import requests

# Fetch HTML
response = requests.get("https://example.com")
html = response.text

# Parse HTML
soup = BeautifulSoup(html, "html.parser")

# Find elements
title = soup.find("title").text
links = soup.find_all("a")
for link in links:
    print(link.get("href"))

# Select with CSS selectors
elements = soup.select(".class-name")
```

Best Practices

Code Style (PEP 8)

```python
```

```python
# Variable naming
user_name = "Luis"  # Use snake_case for variables
UserProfile = None  # Use PascalCase for classes
CONSTANT_VALUE = 3.14  # Use UPPER_CASE for constants

# Line length (max 79 characters)
long_variable = some_function(param1, param2,
                              param3, param4)


# Spacing
x = 5 + 3  # Spaces around operators
def my_function(a, b):  # Space after comma in parameters
    pass

# Comments
# Use comments sparingly, write clear code
# that explains itself
```

DRY Principle (Don't Repeat Yourself)

python

```python
# Bad
print("Luis is 20 years old")
print("Ana is 19 years old")
print("Bob is 22 years old")

# Good
people = [("Luis", 20), ("Ana", 19), ("Bob", 22)]
for name, age in people:
    print(f"{name} is {age} years old")
```

Testing Code

python

```python
# Simple assertion testing
def add(a, b):
    return a + b

assert add(2, 3) == 5, "Add function failed"
assert add(-1, 1) == 0, "Add function failed with negative"

# Using unittest module
import unittest

class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(add(-1, 1), 0)

if __name__ == "__main__":
    unittest.main()
```

Documentation

python

```python
def calculate_compound_interest(principal, rate, time):
    """
    Calculate compound interest.

    Args:
        principal (float): Initial amount
        rate (float): Annual interest rate (as decimal)
        time (float): Time in years

    Returns:
        float: Final amount

    Raises:
        ValueError: If any parameter is negative

    Example:
        >>> calculate_compound_interest(1000, 0.05, 2)
        1102.5
    """
    if principal < 0 or rate < 0 or time < 0:
        raise ValueError("Parameters cannot be negative")
    return principal * (1 + rate) ** time
```

Project Ideas

Beginner Projects

1. **Calculator Program**: Build an interactive calculator with add, subtract, multiply, divide
2. **To-Do List**: Create a command-line task manager with save/load functionality
3. **Guessing Game**: Number guessing game with hints and limited attempts
4. **Password Strength Checker**: Analyze passwords for security criteria
5. **Temperature Converter**: Convert between Celsius, Fahrenheit, Kelvin

Intermediate Projects

1. **Expense Tracker**: Track spending with categories, budgets, and reports
2. **Weather App**: Fetch and display weather data from an API
3. **Web Scraper**: Extract data from websites (with permission)
4. **Chat Bot**: Simple bot that responds to user queries

5. **Stock Price Monitor**: Track stock prices and send alerts

Advanced Projects

1. **Machine Learning Model**: Build and train a classifier using scikit-learn
2. **Web Application**: Create a Flask/Django web app with database
3. **Data Analysis Project**: Analyze large datasets with Pandas and Matplotlib
4. **API Development**: Build a REST API with Flask
5. **Game Development**: Create a game using Pygame

Quick Reference Cheatsheet

String Methods

`upper()`, `lower()`, `capitalize()`, `strip()`, `split()`, `join()`, `replace()`, `find()`, `startswith()`, `endswith()`

List Methods

`append()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()`, `clear()`, `copy()`, `index()`, `count()`

Dictionary Methods

`keys()`, `values()`, `items()`, `get()`, `pop()`, `update()`, `clear()`, `copy()`

Useful Functions

`len()`, `type()`, `range()`, `enumerate()`, `zip()`, `map()`, `filter()`, `sorted()`, `sum()`, `min()`, `max()`

Resources for Further Learning

- **Official Python Documentation**: docs.python.org
- **Real Python**: realpython.com (tutorials and articles)
- **Codecademy**: Learn Python interactively
- **LeetCode/HackerRank**: Practice coding challenges
- **GitHub**: Study open-source projects
- **YouTube**: Numerous free Python courses

Next Steps

1. **Practice coding daily**: Start with simple programs
2. **Build projects**: Apply what you learn to real problems
3. **Read others' code**: Learn from experienced developers
4. **Join communities**: Python subreddits, Discord servers, local meetups
5. **Specialize**: Focus on areas that interest you (web, data science, AI, etc.)

Happy Learning! Python is a journey, not a destination. Keep coding, keep learning, and enjoy the process.